

Cross Modal Data Discovery over Structured and Unstructured Data Lakes

Mohamed Y. Eltabakh
Qatar Computing Research Institute (QCRI)
Doha, Qatar
meltabakh@hbku.edu.qa

Mayuresh Kunjir
Amazon Web Services
Berlin, Germany
mkunjir@amazon.de

Ahmed K. Elmagarmid
Qatar Computing Research Institute (QCRI)
Doha, Qatar
aelmagarmid@hbku.edu.qa

Mohammad Shahmeer Ahmad
Qatar Computing Research Institute (QCRI)
Doha, Qatar
mohammadshahmeerah@hbku.edu.qa

ABSTRACT

Organizations are collecting increasingly large amounts of data for data-driven decision making. These data are often dumped into a centralized repository, e.g., a data lake, consisting of thousands of structured and unstructured datasets. Perversely, such mixture makes the problem of *discovering* tables or documents that are relevant to a user's query very challenging. Despite the recent efforts in *data discovery*, the problem remains widely open especially in the two fronts of (1) discovering relationships and relatedness across structured and unstructured datasets–where existing techniques suffer from either scalability, being customized for a specific problem type (e.g., entity matching or data integration), or demolishing the structural properties on its way, and (2) developing a holistic system for integrating various similarity measurements and sketches in an effective way to boost the discovery accuracy.

In this paper, we propose a new data discovery system, named CMDL, for addressing these two limitations. CMDL supports the data discovery process over both structured and unstructured data while retaining the structural properties of tables. As a result, CMDL is the only system to date that empowers end-users to seamlessly pipeline the discovery tasks across the two modalities. We propose a novel multi-modal embedding representation that captures the similarities between text documents and tabular columns. The model training relies on labeled datasets generated though *weak supervision*, and thus the system is domain agnostic and easily generalizable. We evaluate CMDL on three real-world data lakes with diverse applications and show that our system is significantly more effective for cross-modality discovery compared to the search-based baseline techniques. Moreover, CMDL is more accurate and robust to different data types and distributions compared to the state-of-the-art systems that are limited to only the structured datasets.

PVLDB Reference Format:
Mohamed Y. Eltabakh, Mayuresh Kunjir, Ahmed K. Elmagarmid, and Mohammad Shahmeer Ahmad. Cross Modal Data Discovery over Structured and Unstructured Data Lakes. PVLDB, 16(11): 3377 - 3390, 2023.
doi:10.14778/3611479.3611533

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 16, No. 11 ISSN 2150-8097.
doi:10.14778/3611479.3611533

PVLDB Artifact Availability:
The source code, data, and/or other artifacts have been made available at https://github.com/qcri/CMDL

1 INTRODUCTION

The democratization of data science has resulted in enterprises drowning in their own collected data. Large enterprises routinely have a huge collection of structured and unstructured data in their data repositories, referred to as *data lakes*. Performing *data discovery* where the goal is to find relevant elements (e.g., tables or documents) for a given user's query or an analytical task from such massive and unorganized data lakes is a major challenge faced by data scientists [3, 17, 39, 42, 56]. Data discovery is one of the initial steps of data preparation that plays an important role in various downstream data processing tasks including data exploration, data integration, and data enrichment [28, 32, 33, 55, 62].

Data discovery is emerging as a practical and core problem to modern enterprises as exemplified by the recent interest from both academia and industry [1, 2, 10, 22, 29, 40]. Nevertheless, it remains a challenging problem due to several factors. First, data is collected and published by disparate sources, and thus do not conform to a common format or structure. A vast amount of this data is in the form of unstructured text documents, e.g., research articles, social media posts, emails, news reports, etc. [12, 19]. As a result, both data modalities (structured and unstructured) are equally critical, and any system that overlooks either of them is lame. The second factor is the lack of consistent and sufficient data description and metadata. Therefore, a robust data discovery system must reason about and establish relationships of relatedness using multiple and diverse similarity signals, where each could be possibly weak. Lastly, a data discovery system needs to scale well to very large volumes of data as more organizations and governments are publishing more data for transparency [54].

The majority of existing work limits the discovery process to only the structured portion of data lakes, e.g., [10, 22, 24, 25, 59]. And thus, these systems do not extend to unstructured data and hit the wall of the first challenge mentioned above. In contrast, there are recent efforts trying to extend the discovery process across both structured and unstructured data sets, however they inherit several limitations. For example, some of these systems shift the focus entirely to the unstructured domain by transforming the

structured data to unstructured documents [12, 13, 19]. This way, they demolish the structural properties and cut the way towards having a holistic discovery system across both modalities. Other systems move the two modalities to a third common space where everything is encoded as a set of subject-predicate-object triplets, e.g., [23]. Again, they lose the ability for supporting discoveries such as table joinability and unioinability in addition to encountering high overheads that hinder its scalability.

The goal of this work is to develop a more holistic data discovery system that can: (1) bring large unstructured data into the framework of data discovery, (2) enable analysts to intermix their structured and unstructured discovery tasks in a single pipeline seamlessly, and (3) be robust through novel integration of various content-based and metadata-based similarity signals–this is especially required because unstructured data inherit very different characteristics compared to structured tables.

Motivation Example (Pharmaceutical Usecase): *Pharmaceutical databases such as DrugBank consist of an exhaustive listing of drugs, enzymes, and genes along with their interactions with other drugs or related proteins [4]. Domain experts periodically curate such databases with findings from research literature, MedLine reports [6], FDA MedWatch reports [5], and other sources. Assume our data lake encompasses all these rich data sets, and the analysts are exploring the data in the wild, i.e., without any prior knowledge of the available data sources. Now, an analyst studying enzyme* "Thymidylate Synthase" *would like at first to find documents related to this enzyme (Q1 in Figure 1). The system returns such documents as depicted in the left side of the figure. Then, she poses the second question (Q2) trying to find curated tables in the database related to the 1st document (or sub-document by highlighting sentences of interest). The system should provide table* "Enzyme_Targets" *as an answer. Then, progressing forward with her exploration, she finds another returned document that talks about drug* "Pemetrexed" *and its relation to enzyme* "Thymidylate Synthase". *As such, she poses Q3 to potentially find more tables in the database supporting this relationship. The system should provide the two tables* "Enzyme_Targets" *and* "Drugs" *as high likelihood candidates to Q3. One step further, the analyst would like to know if there are more relationships in the database around this drug. And thus, she raises Q4 and the system provides two potentially joinable tables (i.e.,* "Enzyme_Targets" *and* "Drug_Interactions"*). The former table seems central to this exploration as it has been reached from multiple paths, and hence the analyst issues Q5 to find more related and unionable tables.*

Evidently, the kind of relationships, similarity measures, and sketches needed to be maintained and exploited to answer Question Q1 (within the unstructured data modality) are very different from those needed to answer Questions Q2 and Q3 (across modalities), and also very distinct from those needed to answer Question Q4 (within the structured data modality). To the best of our knowledge, there is no single holistic system that can support such data discovery chain as highlighted above.

Solution overview: We propose a data discovery system, called CMDL[1], that enables the data discovery tasks demonstrated in the motivation example over a data lake of structured and unstructured data sets. To support cross-modal questions such as Q2 and Q3 (from Figure 1), we introduce a novel embedding-based joint representation that brings both unstructured documents and relational tables, more specifically the individual columns in each table, into a common vector space. We propose several optimizations over the model's loss function to achieve scalability and avoid biases as will be explained later. The model that creates these joint representations relies on supervised learning and requires the presence of a labeled training data set–which may not exist in the first place. Therefore, CMDL is equipped with a weakly-supervised training process that integrates multiple similarity signals over documents and columns to construct a labeled training data set. As such CMDL does not require a pre-existing labeled datasets, and hence, easily generalizable to different applications.

To support other types of data discovery questions such as Q1, Q4, and Q5 (from Figure 1), CMDL utilizes several types of signatures and sketches on both the actual data content and the contextual metadata. For example, for structured data, schema information and tables names are types of metadata. Similarly, for unstructured documents, the documents' names and their sources are the metadata. Then, we build appropriate indexes using state-of-the-art techniques on them, e.g., BM25 elastic search index [49] for keyword search and Locality Sensitive Hashing (LSH) index [61] for the set containment operations. While prior data discovery solutions are also based on similar sketches and indexes, e.g., [10, 13, 22, 24, 25], CMDL goes one step beyond by first leveraging them as similarity signals for the weak-supervised training process mentioned above, and second integrating these complementary elements into a single unified discovery system.

The various relationships discovered by the system form an Enterprise Knowledge Graph (EKG) on which a comprehensive query interface is developed. It extends the *SRQL* query engine from Aurum [22] with APIs and retrieval methods specific to CMDL, e.g., for supporting cross-modality discovery tasks.

In summary, we make the following contributions:

(1) Introducing a holistic data discovery system that treats both structured tabular data and unstructured documents as first-class citizens. Within a single discovery pipeline, end-users can seamlessly formulate discovery questions ranging from simple keyword search over either modality to complex discoveries spanning both modalities.
(2) Proposing a novel embedding-based joint representation for multi-modal data sets. The model training relies on weak supervision by combining various types of similarity signals. As such, the system can be adopted even under the absence of labeled data.
(3) Integrating existing sketches with a few alterations to improve the discovery accuracy even for the traditional joinability and unionability tasks by up to 30%.
(4) Evaluating CMDL on three real-world data lakes ranging from drug discovery to ML data augmentation. CMDL is not only more effective in discovering the complex relationships across the unstructured documents and the structured tables, but also more robust for the traditional discovery tasks (e.g., joinability and unionability) over the structured data compared to the state-of-the-art systems.

[1] CMDL–Cross Modal Data Discovery over Structured and Unstructured Data Lakes–is pronounced as kam·dl.

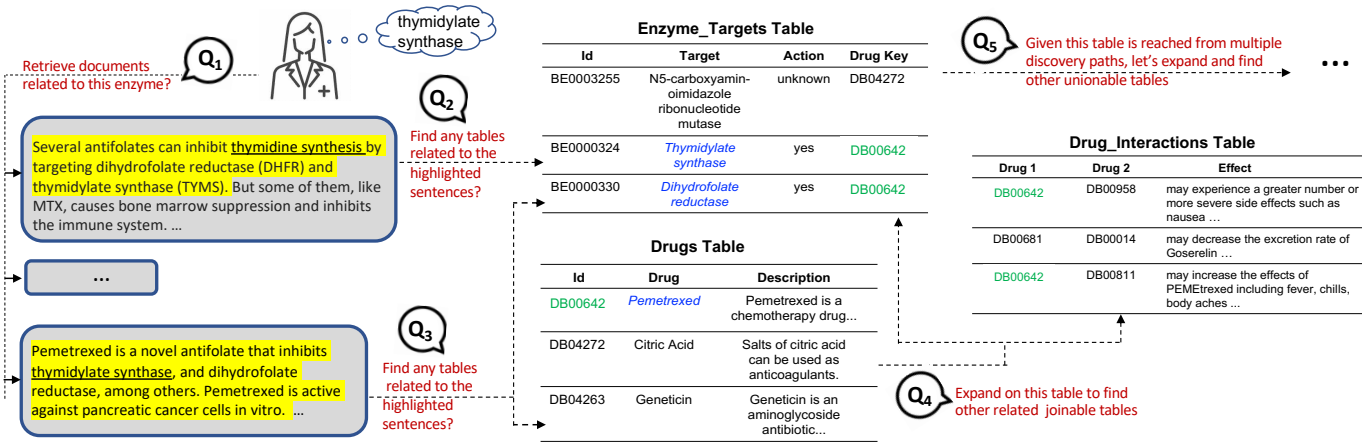


Figure 1: An example data discovery scenario on a lake of structured and unstructured pharmaceutical data.

2 CMDL OVERVIEW

In order to support data discovery pipelines similar to that presented in Figure 1, we need four building block elements. First, the notion of *discoverable elements* (*DE*s) that are the output from the discovery queries. Second, semantics for identifying the relationship between the *DE*s. Third, algorithms and data structures to efficiently discover and store the *DE*s and their relationships. Finally, an expressive query language for expressing the discovery tasks.

2.1 Discoverable Elements and Relationships

CMDL Discoverable Elements. We borrow the concept of Discoverable Elements from Aurum [22], which is the abstract unit of data discovery. In CMDL, we treat each **column** and each **document** as the basic units of discovery for the structured and unstructured data sets, respectively. Based on columns, each **table** in the data lake is also treated as a higher-order DE. The intuition behind the column-level granularity is multifold. (1) Working at this granularity allows the system to scale well to massive datasets, (2) The column-level discovery is already the basic unit of discovery over structured data (e.g., joinability and unionability) [10, 26, 59], and thus it would work seamlessly in the entire CMDL framework, and (3) Columns typically have coherent semantics, e.g., drug names or city names, that embeddings can easily capture (unlike tuples which could have many attributes with very diverse semantics).

For simplicity, we assume each unstructured document is short, i.e., several sentences as illustrated in Figure 1. From a use case and end-user point of view, this assumption is realistic, because otherwise starting a discovery search with a large document, e.g., several pages, is very unfocused and will most probably lead to countless relationships of no interest. Moreover, from the system's backend point of view, this assumption is not restrictive because large documents can still be uploaded as physical units, however, CMDL will (logically) break each document into smaller DE units, e.g., paragraphs, for the discovery purpose.

CMDL DE Relationship Types. In this paper, we emphasize three key relationship types between DEs as motivated by our example in Section 1.

Doc*$_{to}$ *Table *(From Document to Tables).* A Table T with column set $\mathbf{A}$ is related to a text document D iff $\exists A_i \in \mathbf{A}$ s.t. D and A_i are related via overlapping values, semantic similarity, or metadata similarity, each with a relatedness score. The combined scores of the links $D \rightarrow A_i$ represent the strength of the relationship.

Table*$_J$ *Table *(Joinable Tables).* Table T with column set $\mathbf{A}$ is joinable to Table T' with column set $\mathbf{A'}$ iff $\exists A_i \in \mathbf{A}$ and some $A'_j \in \mathbf{A'}$ s.t.: (a) A_i and A'_j have value overlap suggesting syntactic join, or (b) A_i and A'_j have semantic overlap suggesting semantic join.

Table*$_U$ *Table *(Unionable Tables).* Table T with column set $\mathbf{A}$ is unionable to Table T' with column set $\mathbf{A'}$ iff a one-to-one mapping $H : \mathbf{A} \Rightarrow \mathbf{A'}$ exists wherein $\exists h \in H$ s.t. the column pair given by h exhibits name, value, or semantic similarity. The combined similarity score of the column mapping gives the strength of the relation.

2.2 CMDL Architecture

Figure 2 illustrates the overall system's architecture. From left to right, both the text documents and structured tables go through a preprocessing phase. More specifically, each text document goes through an NLP-based pipeline for transformation, which ultimately results in converting a document into a column format consisting of a bag of words. In contrast, each column in the tabular data goes through heuristic-based tagging that labels the column based on the potential relationships and discovery tasks it will participate in. The next step is the *profiler*, which creates various types of sketches and statistics to be leveraged later in the discovery tasks for capturing syntactic and semantic relationships. These sketches then go to the *indexing framework* for building the appropriate indexes according to the individual sketch type (e.g., for metadata and content data, an elastic search index is constructed, whereas for min hash statistics an LSHEnsemble index is constructed).

The sketches created so far by the *profiler* are independent of and agnostic to any relationships between the text documents and the tabular columns. The *Multi-Modal Joint Representation* module is responsible for creating joint sketches that encode such relationships, i.e., related document-column pairs should have similar

LIFTus: An Adaptive Multi-aspect Column Representation Learning for Table Union Search

Ermu Qiu[†], Jun Gao[†*], Yaofeng Tu[‡], and Jingru Yang[§*]
[†]Key Laboratory of High Confidence Software Technologies, CS, Peking University, China [‡]ZTE Corporation,
[§]National Key Laboratory of Data Space Technology and System
qem@stu.pku.edu.cn, gaojun@pku.edu.cn, tu.yaofeng@zte.com.cn, okiyang@pku.edu.cn

***Abstract*—Table union search (TUS) represents a fundamental operation in data lakes to find tables unionable to the given one. Recent approaches to TUS mainly learn column representations for searching by introducing Pre-trained Language Models (PLMs), especially on columns with linguistic data. However, a significant amount of non-linguistic data, notably represented by domain-specific strings and numerical data in the data lake, are still under-explored in the existing methods. To address this issue, we propose LIFTus, an adaptive multi-aspect column representation for table unionable search, where aspect refers to a concept more flexible than data types, so that a single column can exhibit multiple aspects simultaneously. LIFTus aims at combining different aspects of a column (including both linguistic and non-linguistic aspects) to promote the effectiveness and generalization of TUS in a self-supervised manner. Specifically, besides employing PLMs to extract the linguistic aspects from an individual column, LIFTus trains a pattern encoder to learn possible character-level sequential patterns for the column, and builds a number encoder to capture numerical aspects of the column, including the distribution and magnitude features. LIFTus further utilizes a hierarchical cross-attention aided by aspect-relevant statistics to combine these aspects adaptively in producing the final column representations, which are indexed by vector retrieval techniques to achieve efficient search. Extensive experimental results demonstrate that LIFTus has outperformed the current state-of-the-art methods in terms of effectiveness, and achieved much better generalization capability to support unseen data.**

***Index Terms*—Table Union Search, Data Lake, Aspect, Non-Linguistic Column, Generalization**

I. Introduction

With the rapid growth of data from various sources, data lake [1]–[3] becomes an attractive infrastructure to accommodate flexible types of data in their raw form directly. Unlike data warehouses [4], data lakes allow storing data including relational tables (sometimes with metadata unavailable) and other types without transforming into a predefined schema [5]. Thus, a fundamental step for an analytic operation in data lake is to locate the relevant data. TUS is such an operation to identify a set of unionable tables in a data lake that share similar columns with the given query Table [6].

TUS methods attempt to achieve effectiveness, efficiency and generalization. Data instances collected in tables within a data lake often exhibit considerable heterogeneity, encompassing linguistic data (*e.g.* words, long texts, dates), non-linguistic data (*e.g.* domain-specific strings, numerical values), and even their combinations [2], [7], [8]. It is challenging to find the desired unionable target tables effectively. In addition, the column representation learning, a key step in the TUS, needs to capture features of flexible data instances and utilize external knowledge, indicating high time consumption. Moreover, TUS methods should be adaptive to new and evolving data, as the volume of data in the lake grows rapidly. Consequently, a generalized model for column representation is highly desirable.

In the context of the TUS problem, unionability refers to the potential for attributes or tables to be unioned based on shared domain features [9]. Early approaches to TUS have focused on designing rules to measure the unionability between two tables [2], [9], [10], but face challenges in capturing latent linguistic information. With the development of PLMs such as BERT [11] and its variants [12], [13], recent works [14]–[17] have employed these models into TUS, as the data in the lake have flexible format including the textual features [18]. Besides PLMs, knowledge bases (KBs) [19], [20] can also provide information for the determination of unionability [7]. Some studies further notice the roles of the inter-column relationships and compose a token sequence by sampling data from different columns into the BERT-like models, thereby learning the context-aware column representations [14], [15]. The learned column representations are then fed into the vector index for fast similarity search [14]. Some studies calculate pairwise similarities of columns to build a semantic graph for TUS [19], achieving promising performance but at the cost of additional preprocessing and online search time [7].

While PLM-based methods become mainstream in TUS, their preference for linguistic data imposes potential limitations on the performances, especially in the context of the relational tables. We illustrate these issues in Fig. 1. Table A and Table B can be unionable as they both pertain to the entities of washing machines manufactured by the same company and their corresponding columns are unionable, while Table C serves as an example that is prone to model misjudgment. We can see that three tables possess linguistic data in their *Location* and *Description* columns, and Table C also includes the *Color* column as part of the linguistic data. Besides, Fig. 1 illustrates the widespread presence of non-linguistic data in the data lake, including but not limited to identifiers (*e.g. Product Code* column) and numerical data (*e.g. ID* column, *Capacity* column, *Price* column, etc.). For

* Corresponding authors

2025 IEEE 41st International Conference on Data Engineering (ICDE) | 979-8-3315-3603-9/25/$31.00 ©2025 IEEE | DOI: 10.1109/ICDE65448.2025.00165

2375-026X/25/$31.00 ©2025 IEEE
DOI 10.1109/ICDE65448.2025.00165

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:40:17 UTC from IEEE Xplore. Restrictions apply.

example, the strings from the *Product Code* and *Function Abbr* columns (*e.g.* "SDL/JHX/BP/ZH" and "MB65V35E") are not tokens in natural language but highly domain-specific, as they are codes designed by washing machine manufacturers.

Table A

ID	Product Code	Capacity(L)	Function Abbr	Location	Price	Description
20001	EMS75B37mate6	7.5	SDL/JHX/BP/ZH	WalMart	3100	This is a top-loading washing machine that deli...
20002	ES100B35Mate5	10	DDL/JHX/DP	WalMart	2950	Featuring a 10L capacity, it can handle large loa...
20003	ES90B26Mate6	9	SDL/BP/FCR	WalMart	2200	The machine's smart sensors adjust the water...

Table B

ID	Product Code	Capacity(L)	Function Abbr	Location	Price	Description
20981	EMS65B26Mate6	6.5	DDL/DP/DLCJ	Carrefour	1850	With a variety of wash cycles, including a quick...
20982	EB80B37Mate5	8	DDL/DP	Carrefour	2100	Its durable stainless steel drum is gentle on fab...
20983	ES95B36PLUS5	9.5	SDL/BP/FCR/ZH	Carrefour	3500	Equip with a self-cleaning function, eliminating...

Table C

ID	Product Code	Power(W)	Color	Location	Price	Description
301	MB65V35E	340	White	WalMart	1900	The machine's sleek design and compact size m...
302	MB75V35E	400	Silver	WalMart	2150	For those with allergies, the UltraClean 3000's...
303	MB65V55E	360	Black	WalMart	2350	With a quiet operation and a 5-year warranty, t...

Fig. 1. An example of data lake. In this data lake, Table A and Table B are unionable with each other, while Table C is easily misjudged as being unionable with Table A. This misjudgment often stems from an inadequate understanding of non-linguistic columns (*e.g. ID*, *Product Code*, *Function Abbr*), which are widely distributed across the data lake and play a crucial role in determining the unionability of tables. The metadata present here is for easy understanding, but not used in the following processing.

We illustrate the issues faced by the existing PLM-based methods. First, the domain-specific strings, represented by commonly used identifiers, fall outside the typical token set in the PLMs [11]–[13], and cannot be captured by PLMs effectively. Taking Fig. 1 as an example, as the token sets in the PLMs do not cover the values in *Product Code* column and *Function Abbr*, PLMs handle them using the frequent sub-strings, like BPE [21], computed from the large common corpus, which may be different from the key sub-parts in these domain-specific strings. We can see that the similarity between "EMS100B37mate6" and "EMS65B26Mate6" should be significantly greater than that between "EMS100B37mate6" and "MB65V35E". However, PLMs may decompose the string into different parts (*e.g.* "E", "MS1"), which damages the understanding of the domain-specific string. It is imperative to employ character-level sequential modeling for domain-specific strings to capture their similarities.

Second, the accurate perception of the numerical values to match columns is an open problem [22]. It is a live issue for PLMs to adequately understand numerical values [23]. PLMs' token sets can not cover infinite numerical values distributed in continuous space. Thus, the PLM-based methods [14]–[16] struggle to fully understand real numbers, which leads to a loss in the similarity computation over arbitrary numbers in real-life applications. The set-coverage-based methods [9] may yield Jaccard similarity scores nearly zero (*e.g.* {7.5, 10, 9} in *Capacity* column in Table A, {6.5, 8, 9.5} in *Capacity* column in Table B in Fig. 1), despite the fact that these numbers differ only slightly. We also notice that there has been research on encoding numerical types in tables [24]. However, there remains a significant gap between encoding individual numbers and encoding entire numerical columns in TUS, where the latter needs to capture the crucial criteria to determine the similarity among columns, including distribution and magnitude features.

Third, columns may include combined features, and relying solely on data-type-specific approaches may be too coarse to accurately capture column features. As we can see from Fig. 1, some columns in a data lake are rich in semantic information, characterized by a high proportion of natural language words, such as the *Description* columns, which rely more on external corpora or PLMs. In contrast, the *Function Abbr* columns in Table A and Table B should guide the model to prioritize character-level similarity during unionable search. Additionally, some columns are predominantly numerical, such as the *ID* and *Price* columns across the three tables. In such a case, the determination should focus on the distribution and magnitude features of the numbers. Moreover, certain columns contain a mixture of numerical data and non-language strings, such as the *Product Code* columns in the three tables. We can see that even if we can obtain the data type for the column, the data-type-specific methods are not sufficient to capture the flexible features inside the column.

To overcome the aforementioned challenges and issues of the existing methods, we make the following design choices. (1) We introduce a notation of *column aspect*, which is finer than column data type, as each column has only one data type but with multiple aspects. Thus, instead of invoking one method after determining the column data type, we apply multiple aspect extractors on one column simultaneously, so as to handle the different forms of the same typed column or mixed-typed data. We also expect that the full exploration of aspects in an individual column can enhance the model's generalization to unseen data. (2) We do not solely rely on PLM to handle all kinds of columns, but introduce effective methods to capture distinct features in columns, including the linguistic data, domain-specific strings, and numerical values. For aggregating these aspects, we design a learning method rather than heuristic rules to enable robust adaptation to scenarios involving tables with diverse information. (3) We mainly study the exploration of features in one column, and leave the inter-column relationships into future work, since different features inside one single column, we believe, are still underexplored. (4) The method will yield the column embedding, instead of the pairwise similarity model in order to leverage the existing vector index like HNSW (Hierarchical Navigable Small World) [25] to improve the efficiency of online search.

This paper proposes **LIFTus**, an adaptive mu**l**ti-aspect column representat**i**on **f**or **t**able **u**nionable **s**earch. The contributions are as follows:

- We design methods to capture the key non-linguistic aspects in LIFTus, including a pattern encoder for domain-specific strings to infer similarities between strings, and a number encoder to convert the numerical aspect into embedding that preserves the distribution and magnitude features.
- We devise a hierarchical cross-attention integrator, to adaptively combine different aspects of columns, in which

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:40:17 UTC from IEEE Xplore. Restrictions apply.

the statistics indicating the weights of different aspects are extracted and fed into a hierarchical cross-attention framework to produce the column representations.
- We conduct experiments on four real-world datasets, and the results indicate that LIFTus has achieved a 3.2% improvement in effectiveness on average (**4.7% improvement** for 2 large datasets on average) over the second to the best model at a similar offline preprocessing time cost. The results also show much better generalization capability of LIFTus, which is trained on a small dataset but can be applied to a large dataset, thereby saving training time. For example, after training on SANTOS small only, LIFTus can achieve even better performance on SANTOS large than Starmie, which takes **5.6X** more time training on SANTOS large directly. The code is available on https://github.com/MsiQue/LIFTus.

II. Related Work

In this section, we first review the advances of approaches to TUS, then introduce recent progress in a more generalized table representation learning, and finally discuss the methods used to process non-linguistic data within tables.

A. Table Union Search

TUS is a frontier research topic and has witnessed rapid advancements recently [2], [7], [10], [14]. We review the existing methods from various design choices, ranging from the measurement of the unionable table, different forms of external knowledge, the consideration of inter-column relationships, and training strategies.

There are explicit and implicit measurements for the unionable table. Nargesian et al. [9] have introduced the concept of TUS, and proposed three aspects of unionability. Bogatu et al. [10] have introduced D^3L, extending the determination of unionability to a five-dimensional model. Other recent works, including SANTOS [7] and Starmie [14], learn the unionable tables in an implicit way, in which the previous factors are constructed as a column relation graph or fed into PLM.

Different external knowledge can be incorporated into the TUS. The early works [2], [10] mainly utilize external knowledge bases [19], [20] or word embeddings [26]. SANTOS [7] leverages both existing knowledge bases [19] and a synthesized one, generated by exploiting the inherent knowledge within the data lake. Starmie [14] relied on PLMs [11]–[13], which are based on the transformer architecture [27] to learn the attention weights between token pairs.

The inter-column relationships are considered by SANTOS [7] and Starmie [14]. They assume that the neighboring columns provide a context for an individual column. Specifically, SANTOS acquires semantic relationships using the knowledge base, and Starmie vectorizes each column by concatenating tokens sampled from neighboring columns and feeding them into a language model.

The model can be trained in a supervised or self-supervised fashion. D^3L [10] treats the determination of unionable tables as a binary classification problem in a supervised manner. To reduce the high cost of manual labeling, Watchog [28] and Starmie [14] have shifted to the self-supervised fashion, allowing researchers to expand the column encoder for various downstream tasks.

The existing methods bring the following inspiration to LIFTus. LIFTus follows a self-supervised fashion to train a column encoder for TUS with sufficient factors considered. In addition, LIFTus chooses the PLM to handle the linguistic data as a part of producing column representations. LIFTus puts off the inter-column relationship to the future work, as we observe that the column features are not fully explored in the current data preprocessing.

B. Table Representation Learning

TUS is a specific downstream task of the generalized table representation learning, which can support other tasks like column classification, data integration, etc. We can roughly categorize the existing methods into basic neural network based approaches [29]–[32], graph representation learning based approaches [33]–[35], and pretrained model based approaches [22], [36]–[40]. Among them, Sherlock [29] employs a multi-input deep neural network to integrate various feature categories and learns column embeddings in a supervised manner. SATO [30] extends Sherlock [29] by incorporating inter-column information and leverages Latent Dirichlet Allocation (LDA) [41]. EmbDI [33] and HierGAT [34] learn embeddings for both columns and rows by utilizing graph-based random walk and attention mechanism, respectively. Pretrained model based approaches can mainly be divided into two categories. One is to use tables as a corpus to pre-train the models [38]–[40], and the other is to directly utilize PLMs to understand tables for solving downstream tasks [22], [28], [36], [37].

C. Non-linguistic Column Understanding

Domain-specific String. Existing works handle the domain-specific strings by extracting their formatting rules or regular expressions. Cornet [42], [43] leverages user examples to automatically learn formatting rules for a specific column. Bogatu et al. [10] establish a collection of rules to transform columns into a set of regular expressions and measure the similarity of two columns using the Jaccard similarity. Sherlock [29] focuses on the statistics of characters in strings (*e.g.* mean, max, sum, etc.), while the order of characters within a string is not extensively explored in computing similarity between strings. In the context of TUS, we actually do not require explicit regular expressions for strings, and an implicit model is sufficient for the similarity among strings.

Numerical Data. Some works measure the correlation between two numerical columns using statistics. STEER [44] proposes a steered-labeling approach to handle numerical columns effectively with Earth Mover's Distance for similarity assessment, which leverages non-numerical column labels to guide context-aware clustering. Bogatu et al. [10] measure the correlation by computing the Kolmogorov-Smirnov (KS)

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:40:17 UTC from IEEE Xplore. Restrictions apply.



Semantics-aware Dataset Discovery from Data Lakes with Contextualized Column-based Representation Learning

Grace Fan
Northeastern University
United States
fan.gr@northeastern.edu

Jin Wang
Megagon Labs
United States
jin@megagon.ai

Yuliang Li
Megagon Labs
United States
yuliang@megagon.ai

Dan Zhang
Megagon Labs
United States
dan_z@megagon.ai

Renée J. Miller
Northeastern University
United States
miller@northeastern.edu

ABSTRACT

Dataset discovery from data lakes is essential in many real application scenarios. In this paper, we propose Starmie, an end-to-end framework for dataset discovery from data lakes (with table union search as the main use case). Our proposed framework features a contrastive learning method to train column encoders from pre-trained language models in a fully unsupervised manner. The column encoder of Starmie captures the rich contextual semantic information within tables by leveraging a contrastive multi-column pre-training strategy. We utilize the cosine similarity between column embedding vectors as the column unionability score and propose a filter-and-verification framework that allows exploring a variety of design choices to compute the unionability score between two tables accordingly. Empirical results on real table benchmarks show that Starmie outperforms the best-known solutions in the effectiveness of table union search by 6.8 in MAP and recall. Moreover, Starmie is the first to employ the HNSW (Hierarchical Navigable Small World) index to accelerate query processing of table union search which provides a 3,000X performance gain over the linear scan baseline and a 400X performance gain over an LSH index (the state-of-the-art solution for data lake indexing).

PVLDB Reference Format:
Grace Fan, Jin Wang, Yuliang Li, Dan Zhang, and Renée J. Miller. Semantics-aware Dataset Discovery from Data Lakes with Contextualized Column-based Representation Learning. PVLDB, 16(7): 1726 - 1739, 2023.
doi:10.14778/3587136.3587146

PVLDB Artifact Availability:
The source code, data, and/or other artifacts have been made available at https://github.com/megagonlabs/starmie.

1 INTRODUCTION

The growing number of open datasets from governments, academic institutions, and companies have brought new opportunities for innovation, economic growth, and societal benefits. To integrate and analyze such datasets, researchers in both academia and industry have built a number of dataset search engines to support the application of dataset discovery [3, 7, 17, 19, 33, 40, 45]. One popular example is Google's dataset search [3] which provides keyword search on the metadata. However, for open datasets, simple keyword search might suffer from data quality issues of incomplete and inconsistent metadata across different datasets and publishers [1, 16, 41, 42]. Thus it is essential to support *table search* over open datasets, and more generally data lake tables (including private enterprise data lakes), to boost dataset discovery applications, such as finding related tables, domain discovery, and column clustering.

Finding related tables from data lakes [11, 25, 39, 46, 57] has a wide spectrum of real application scenarios. There are two sub-tasks of finding related tables, namely table union search and joinable table search. In this paper, we mainly focus on the problem of *table union search*, which has been recognized as a crucial task in dataset discovery from data lakes [2, 24, 39, 41, 42, 57, 61]. Given a query table and a collection of data lake tables, table union search aims to find all tables that are unionable with the query table. To determine whether two tables are unionable, existing solutions first identify all pairs of unionable columns from the two tables based on column representations, such as bag of tokens or bag of word embeddings. They then devise some mechanism to aggregate the column-level results to compute the table unionability score.

State-of-the-art: Early work on finding unionable tables used table clustering followed by simple syntactic measures such as the difference in column mean string length and cosine similarities to determine if two tables are unionable [4]. Table union search [42] improved on this by applying a rich collection of column representations including syntactic, semantic (leveraging ontologies), and natural language (based on word-embeddings) column representations. Two important innovations of this work were the modeling of data lake context to create an *ensemble unionability score* which models the surprisingness of a score given the score distributions within a data lake and the use of LSH indices to make table union search fast over large data lakes [42]. More recently D^3L [2] added additional column representations based on regular expression matching and SANTOS [24] added to the column representations, representations of binary relationships. In parallel to these search-based approaches, the mighty hammer of deep learning has been applied to the problem of column matching (determining the semantic type of a column) [22, 56]. Since these

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 16, No. 7 ISSN 2150-8097.
doi:10.14778/3587136.3587146

Table A:

Name	Mode of Travel	Purpose	Destination	Day	Month	Year	Expense
Philip Duffy	Air	Regional Meeting	London	10	April	2019	189.06
Jeremy Oppenheim	Taxi	Exchange Visit	Ottawa	30	Jul	2019	8.08
Mark Sedwill	Air	Evening Meal	Bristol	02	September	2019	50

Table B:

Name	Date	Destination	Purpose
Clark	23/07	France	Discuss EU
Gyimah	03/09	Belgium	Build Relations
Harrington	05/08	China	Discuss Productivity

Table C:

Bird Name	Scientific Name	Date	Location
Pine Siskin	Carduelis Pinus	2019	Ottawa
American Robin	Turdus migratorius	2019	Ottawa
Northern Flicker	Colaptes auratus	2019	London

Figure 1: An example of table union search on Open Data.

approaches are supervised, they can only be applied to finding a limited set of semantic types (78 in their experiments), and while not a general solution for unionability in data lakes, they can be used in an offline fashion to find unionable tables containing the types on which they are trained.

However, there are still plenty of opportunities to further improve the performance of table union search. One important issue is to learn sufficient contextual information between columns in tables so as to determine the unionability. This point can be illustrated in the following motivation example.

Example 1.1. Figure 1 shows an example of finding unionable tables. Given the query Table A, existing approaches first find unionable columns. In this example, the column `Destination` in Table A will be deemed more unionable with `Location` from Table C than with `Destination` from Table B. This is because the syntactic similarity score, e.g. overlap and containment Jaccard, between the two `Destination` columns is 0; while the average word embedding of cities (Table A) is also not as close to that of nations (Table B). Similarly, if an ontology is used, Table A and Table C shares the same class while the values in B are in different (though related) classes. Meanwhile, looking at the tables as a whole we observe that Table A is actually irrelevant to Table C. But as existing solutions only look at the pair of single columns when calculating column unionability score, the columns `Year/Date` and `Destination/Location` of the two tables might be wrongly aligned together. Even techniques that look at relationships [24] can be fooled by the value overlap in this relationship and determine the relationship `Year-Destination` in Table A to be unionable with `Date-Location` in Table C. This kind of mistake can be avoided by looking at a table's context, i.e. information carried by other columns within a table. Looking at the table as a whole, a method should be able to recognize that the `Year` in Table A is part of a travel date while in Table C it is the date of discovery of a bird; and `Destination` in Table A refers to the cities to which the officers are traveling; whereas `Location` in Table C is the city where a bird is found.

From the above example, we focus on the following challenges in proposing a new solution. Firstly, it is essential to learn richer semantics of columns based on natural language domain. To this end, we require a more powerful approach to learn the column representation so as to capture richer information instead of relying on simple methods like the average over bag of word embeddings utilized in previous studies [2, 14] or even the similarity of the word embedding distributions [42]. Secondly, we argue that it is crucial to utilize the contextual information within a table to learn the representation of each column, which is ignored by previous studies. Even proposals for capturing relationship semantics do not use contextual information to learn column representations [24]. Finally, due to the large volume of data lake tables, it is also a great challenge to develop a scalable and memory-efficient solution.

We propose Starmie, an end-to-end framework for dataset discovery from data lakes with table union search as the main use case. Starmie uses pre-trained language models (LMs) such as BERT [13] to obtain semantics-aware representations for columns of data lake tables. While pre-trained LMs have been shown to achieve state-of-the-art results in table understanding applications [12, 31, 47], their good performance heavily relies on high-quality labeled training data. For the problem setting of table union search [41, 42], we must come up with a fully unsupervised approach in order to apply pre-trained LMs to such applications, something not yet supported by previous studies. Starmie addresses this issue by leveraging *contrastive representation learning* [10] to learn column representations in a **self-supervised manner**. An innovation of this approach is to assume that two randomly selected columns in a data lake can be used as negative training examples. For positive examples, we propose and use novel data augmentation methods. The framework defines a learning objective that connects the same or similar columns in the representation space while separating distinct columns. As such, Starmie can apply the pre-trained representation model in downstream tasks such as table union search without requiring any labels. We also propose to combine the learning algorithm with a novel multi-column table transformer model to learn *contextualized* column embeddings that model the column semantics depending on not only the column values, but also their context within a table. While a recent study SANTOS [24] can reach a similar goal by employing a knowledge base, our proposed methods can automatically capture such contextual information from tables in an unsupervised manner without relying on any external knowledge or labels.

Based on the proposed column encoders, we use cosine similarity between column embeddings as the column unionability score and develop a bipartite matching based method to calculate the table unionability score. We propose a filter-and-verification framework that enables the use of different indexing and pruning techniques to reduce the number of computations of the expensive bipartite matching. While most previous studies employed LSH index to improve the search performance, we also make use of HNSW (Hierarchical Navigable Small World) index [36] to accelerate query processing. Experimental results show that HNSW can significantly improve the query time while only slightly reducing the MAP/recall scores. Besides table union search, we further conduct two case studies to show that Starmie can also support other dataset discovery applications such as joinable table search and column clustering. We believe these results show great promise in the use of contextualized, self-supervised embeddings for many table understanding tasks.

Our contributions can be summarized as the following.

- We propose Starmie, an end-to-end framework to support dataset discovery over data lakes with table union search as the main use case.
- We develop a contrastive learning framework to learn contextualized column representations for data lake tables without requiring labeled training instances. Starmie achieves an improvement of 6.8% in both MAP and recall compared with the best state-of-the-art method, with a MAP of 99%, a significant margin compared with previous studies.

- We design and implement a filter-and-verification based framework for computing the table-level unionability score which can accommodate multiple design choices of indexing and pruning to accelerate the overall query processing. By leveraging the HNSW index, Starmie achieves up to three orders of magnitude in performance gain for query time relative to the linear scan baseline.
- We conduct an extensive set of experiments over two real world data lake corpora. Experimental results demonstrate that the proposed Starmie framework significantly outperforms existing solutions in effectiveness. It also shows good scalability and memory efficiency.
- We further conduct case studies to show the flexibility and generality of our proposed framework in other dataset discovery applications.

2 OVERVIEW

2.1 Problem definition

A data lake consists of a collection of tables $\mathcal{T}$. Each table $T \in \mathcal{T}$ consists of several columns $\{t_1, \ldots, t_m\}$ where each column t_i can be from different domains. Here m is the number of columns in table T (denoted as $|T| = m$). We will use the notation T to denote both the table and its set of columns if there is no ambiguity. To determine the unionability between two columns, following previous studies, we employ *column encoders* to generate the representations of columns. Then the *column unionability score* can be computed to measure the relevance between those representations. A column encoder $\mathcal{M}$ takes a column t as input and outputs $\mathcal{M}(t)$ as the representation. Given two columns t_i and t_j, the column unionability score is computed as $\mathcal{F}(\mathcal{M}(t_i), \mathcal{M}(t_j))$, where $\mathcal{F}$ is a scoring function between two column representations.

Based on the column unionability scores, we compute the table unionability score between two tables, which is obtained by aggregating the column unionability scores introduced above. Given two tables S and T, we define a table unionability scoring mechanism as $U = \{\mathcal{F}, \mathcal{M}, \mathcal{A}\}$, where $\mathcal{M}$ and $\mathcal{F}$ are the column encoder and scoring function for two column representations, respectively. Here $\mathcal{A}$ is a mechanism to aggregate the column unionability scores between all pairs of columns from the two tables. We will introduce the details of $\mathcal{A}$ later in Section 4.

Following the above discussions, we can formally define the table union search problem as a top-k search problem as Definition 2.1:

Definition 2.1 (Table Union Search). Given a collection of data lake tables $\mathcal{T}$ and a query table S, top-k table union search aims at finding a subset $\mathcal{S} \subseteq \mathcal{T}$ where $|\mathcal{S}| = k$ and $\forall T \in \mathcal{S}$ and $T' \in \mathcal{T} - \mathcal{S}$, we have $U(S, T) \geq U(S, T')$.

2.2 System architecture

Figure 2 shows the overall architecture of Starmie that solves table union search in two stages: offline and online.

During the offline stage, Starmie pre-trains a column representation model that encodes columns of data lake tables into dense high-dimensional vectors (i.e., column embeddings). Then, we apply the trained model to all data lake tables to obtain the column embeddings via model inference. We store the embedding vectors in

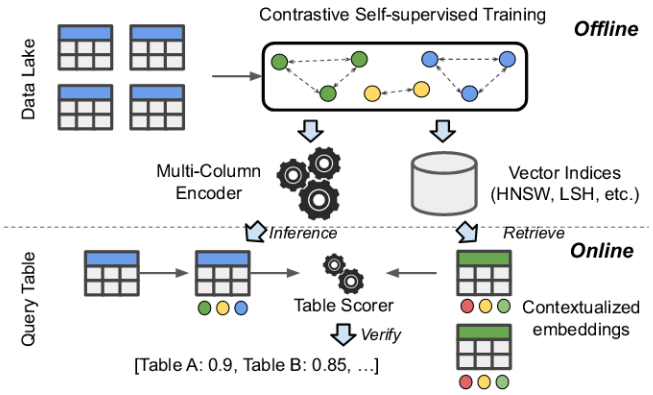


Figure 2: During the offline phase, Starmie **pre-trains a multi-column table encoder using contrastive learning and stores the embeddings of data lake columns in vector indices like HNSW. During online processing,** Starmie **retrieves candidate tables with similar contextualized column embeddings then verifies their table-level unionability scores using column alignment algorithms.**

efficient vector indices for online retrieval. A key challenge for the offline stage is to train high-quality column encoders that capture the semantics of tabular data. In Starmie, we follow a recent trend [12, 31, 47] of table representation learning that encodes tabular data using pre-trained language models (LMs). Pre-trained LMs have achieved state-of-the-art performance on table understanding tasks such as column type and relation type annotation [47]. However, the good performance of pre-trained LMs requires fine-tuning on high-quality labeled datasets, which are always not available in table search applications such as table union search. Using pre-trained LMs off-the-shelf is also problematic as the column embeddings cannot capture (ir-)relevance between columns or the contextual information within tables. To this end, in Section 3, we propose a contrastive learning framework for learning high-dimensional column representations in fully unsupervised manner. We combine the framework with a multi-column table model that captures column semantics from the column values while taking the table context into account. Then we apply the column encoder to all tables to convert each table into a collection of embedding vectors.

During the online stage, given an input query table, we retrieve a set of candidate tables from the vector indices by searching for data lake column embeddings of high column-level similarity with the input columns. Starmie then applies a verification step for checking and ranking the candidates for the top-k tables with the highest table-level unionability scores. The first challenge for the online stage is how to efficiently search for unionable columns. This is not a trivial task due to the massive size of data lakes. We address this challenge by allowing different design choices of state-of-the-art high-dimensional vector indices. Yet another challenge is designing a table unionability function that can effectively aggregate the column unionability scores. As in other studies, we employ weighted bipartite graph matching. To address its limitation of high computation complexity, we introduce a novel algorithm to reduce the number of expensive calls to the exact matching algorithm by deducing lower and upper bounds of the matching score (Section 4).

TabSketchFM: Sketch-based Tabular Representation Learning for Data Discovery over Data Lakes

Aamod Khatiwada[⋆▶] Harsha Kokel[⋆] Ibrahim Abdelaziz Subhajit Chaudhury Julian Dolby
Northeastern University *IBM Research* *IBM Research* *IBM Research* *IBM Research*
khatiwada.a@northeastern.edu harsha.kokel@ibm.com ibrahim.abdelaziz1@ibm.com subhajit@ibm.com dolby@us.ibm.com

Oktie Hassanzadeh Zhenhan Huang[▶] Tejaswini Pedapati Horst Samulowitz Kavitha Srinivas
IBM Research *Rensselaer Polytechnic Institute* *IBM Research* *IBM Research* *IBM Research*
hassanzadeh@us.ibm.com huangz12@rpi.edu tejaswinip@us.ibm.com samulowitz@us.ibm.com kavitha.srinivas@ibm.com

2025 IEEE 41st International Conference on Data Engineering (ICDE) | 979-8-3315-3603-9/25/$31.00 ©2025 IEEE | DOI: 10.1109/ICDE65448.2025.00118

***Abstract*—Enterprises have a growing need to identify relevant tables in data lakes; e.g. tables that are unionable, joinable, or subsets of each other. Tabular neural models can be helpful for such data discovery tasks. In this paper, we present TabSketchFM, a neural tabular model for data discovery over data lakes. First, we propose novel pre-training: a sketch-based approach to enhance the effectiveness of data discovery in neural tabular models. Second, we finetune the pretrained model for identifying unionable, joinable, and subset table pairs and show significant improvement over previous tabular neural models. Third, we present a detailed ablation study to highlight which sketches are crucial for which tasks. Fourth, we use these finetuned models to perform table search; i.e., given a query table, find other tables in a corpus that are unionable, joinable, or that are subsets of the query. Our results demonstrate significant improvements in F1 scores for search compared to state-of-the-art techniques. Finally, we show significant transfer across datasets and tasks establishing that our model can generalize across different tasks and over different data lakes.**

I. Introduction

Enterprises store critical data in *data lakes*, large repositories of tabular data, for both governance and analytics [1]. It is essential to effectively find relevant tables (e.g., joinable, unionable, subsets) within lakes for reporting, decision-making, statistical analysis, and more [2], [3]. For instance, *Unionable table search* helps to augment an existing table with new rows, and to enrich analytics over it [4]. *Joinable table* search is useful to run analytics that needs data in columns from multiple tables, e.g., to identify how many customers in some region purchased a product due to an email campaign. Also, identifying *subsets* or *supersets* of a table is essential, for example, when searching for potential copies of personal data to comply with the General Data Privacy Regulation of the European Union[1]. Motivated by such use cases, we focus on the problem of identifying tables from the data lakes that can be *unionable, joinable, and/or subsets of each other* [4]–[6]. Specifically, in this work, we propose a pre-trained tabular model that could help in enhancing the effectiveness of data discovery techniques.

⋆ Joint first-authors ▶ Work done while at IBM Research.

[1]https://gdpr-info.eu

Central to the problem of identifying relevant tables in the data lakes is identifying similar columns in tables. However, the columns in a table could be ambiguous. For instance, a column called `Age` is different when it is in a table about buildings from when it is in a table about people. Furthermore, values within a column `Age` play a key role in determining semantics; `Age` of buildings in New York are unlikely to match `Age` of historical buildings in Egypt. Contextualizing columns could be useful in resolving such ambiguities. Language Models (LMs) are shown to be successful in contextualizing words in a sentence [7]. Consequently, we use them to contextualize columns and use such contextualization to determine similar columns or tables.

Existing pre-trained tabular models [8]–[11] based on LMs [7] do not focus on identifying relevant tables. Instead, they focus on language-style tasks such as fact checking [10], question answering [12], converting natural language to SQL [11], table cell content population [8], and table metadata prediction [8], [9]. In addition, input to these models is the actual cell values or a row because of the focus on language-style tasks. However, this has two limitations for understanding relevant tables: (a) only a small number of values can be passed in as input because of severe limits on token length in encoder-only models that can generate embeddings for search (e.g. 512 tokens in BERT [7]), and this potentially limits the information available to the model to understand the input table [2] ; (b) treating numerical values in the table as text tends to lose their semantics; it may be better to pass numerical information directly as vectors into the model.

To address these limitations on input length and numerical representation, we introduce a novel transformer-based tabular model—*TabSketchFM*—exploiting both table metadata and

[2] Decoder only models have larger context lengths. However, it has been difficult until recently to use decoder-only models for embeddings [13]. Moreover, these models are expensive to deploy and provide only a minimal improvement in accuracy over encoder-only models such as BERT. This is why embeddings for search are mostly based on BERT-style models. Further, even with large context lengths, it is not practical to serialize tables with millions of rows as sentences.

2375-026X/25/$31.00 ©2025 IEEE
DOI 10.1109/ICDE65448.2025.00118

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:39:19 UTC from IEEE Xplore. Restrictions apply.

cell values that creates different sketches over them instead of linearizing cell values. Specifically, we abstract column cell values using data sketches, as has been used in the data discovery literature (e.g., MinHash sketches [14], and numerical sketches [15]). Such sketches capture tabular features, and we input them into the neural models. Inputting numerical sketches bypasses feeding table cell values to the model, addressing the token limit issue, as well as the numerical representation of cell values.

After pretraining using sketches, we finetune the models for individual data discovery tasks such as union, join, and finding subsets. The embeddings given by the finetuned models are then used for the respective retrieval tasks. While some in the literature [16] have used self supervision for fine tuning, self supervision applies better for some tasks such as unions compared to say joinability. To finetune models for the data discovery tasks, we used a suite of 8 benchmarks across the 3 tasks *LakeBench* that we have open sourced for use [17]. On the fine tuning tasks, *TabSketchFM* outperformed other neural methods, by as much as 55% on F1. We then used the fine tuned models for search tasks, and showed that *TabSketchFM* outperforms the state-of-the-art neural and traditional methods by as much as 70% on F1.

Importantly, we also compare the performance of specialized traditional and neural models for data discovery against an off-the-shelf pretrained model that encodes column values as a single sentence. To our surprise, the off-the-shelf model performs much better than existing systems - in fact, for certain tasks such as union, it is sufficient to use that model for discovering unionable tables; our tabular specific model does worse, as do all other models that target unionability. The insight from this finding is that while some data discovery tasks need values to match across columns (e.g. join or subsets), for union, it is sufficient when columns have the same semantic meaning; indeed there is no need for any value overlap. Sentence embeddings capture those semantics well. Because certain tasks seem to benefit from these sentence embeddings (e.g. union), and other do not (e.g., subset), we propose a way to concatenate the embeddings from sentence encoders with tabular model embeddings, and show that one can augment tabular embeddings with value embeddings to improve performance, at least in some cases.

In summary, our contributions are as follows:

- **A novel tabular pretrained model.** We are the first to present a sketch-based tabular pretrained model, *TabSketchFM*, which contextualizes columns based on their data sketches and tabular metadata. The sketches help to better handle input token limits and numerical cell values. To achieve this, we modified the text-based transformer architecture to take in numerical and MinHash sketches as input. A purely sketch based approach though misses the opportunity to include table semantics from column values. Given that the inputs to the transformer architecture in our system are numeric vectors, it is difficult to combine these numeric inputs with tokens derived from cell values. We therefore combine the sketch based model embeddings with column value embeddings from existing pretrained sentence transformer models. We show that this combination boosts performance on some data discovery tasks.
- **Finetuning models for downstream tasks.** We develop cross-encoders from pretrained TabSketchFM, which we fine-tune for a range of downstream tasks from the LakeBench collection [17]. We fine tune not only *TabSketchFM* but other tabular pretrained models to see if they can perform well on these downstream tasks and present the comparision.
- **Detailed ablation study.** We assess the effectiveness of different sketches for identifying different types of table relevance. Our study presents interesting insights such as Minhash sketches of column values are crucial for identifying joins, but numerical sketches are crucial for identifying subsets; once again highlighting the value of using sketches for table representation.
- **Demonstration of TabSketchFM on search.** We compared the performance of *TabSketchFM*'s fine tuned models against neural and non-neural baselines. In the case of join search, *TabSketchFM* outperformed the nearest systems geared for join search by almost 70% in F1 scores, but tellingly, the sentence encoding model outperformed existing systems as well by 63%. By combining the two embeddings (from TabSketchFM and sentence encodings), we gain almost 73% over other systems in F1.
- **Illustrate generalization.** Finally, we illustrate the main advantage of learning a pretrained foundational model by showing the generalization of search performance across tasks. For example, we fine tune the pretrained model on join identification on one dataset and use it for union search on another dataset. Our evaluation shows that our finetuned models generalize well.

II. Related Work

Tabular Pretrained Models. Neural models are pretrained on tabular datasets to either recover masked tokens in the table or to detect corrupt values [8]–[11]. [18] surveys neural models for tabular data representation and their applications. [8] combined the Masked Language Model (MLM) objective from BERT [7] with a novel Masked Entity Recovery to train TURL. [10] use MLM with novel Cell-level Cloze and Table Context Retrival for TUTA. [11] used Masked Column Prediction and Cell Value Recovery for TABERT while [9] repurposed ELECTRA's objective function [19] for TABBIE. These models are finetuned and evaluated for different downstream tasks such as table metadata prediction, table content population, fact-checking, question answering, semantic parsing, and so on. For example, TABERT is evaluated on neural semantic parsing; TABBIE for column, row, and column type population tasks; and TUTA is finetuned and evaluated for cell and table type classification tasks. These downstream tasks take either a table or a table with a question as input; they do not consider the inter-table tasks such as dataset discovery, which we consider in this paper. Furthermore, they adopt

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:39:19 UTC from IEEE Xplore. Restrictions apply.

serialization techniques from LMs and represent table values as sentences, whereas we represent using sketches.

Dataset Discovery Methods. Different methods such as keyword search [20], [21] and table search [22]–[24] have been used for the dataset discovery over data lakes. Table Union Search (TUS) [4] presented finding top-k data lake tables that are unionable with a query table. They determine the column unionability using three statistical tests based on value overlap, semantic overlap, and word embedding similarity, and aggregate them to infer table unionability. D^3L [25], adopted value overlap and word embedding measures from TUS and added three additional measures to measure column unionability: numerical column distributions, column header similarity, and regular expression matching. SANTOS [6] considers the relationships between the column pairs in both query and data lake tables. Fan et el. [16] developed a contrastive-learning-based table union search technique called Starmie that captures the context of the whole table in the form of column embeddings. AutoTUS [26] searches for unionable tables by contextualizing entire tables with embeddings of the relationship between the column pairs. Furthermore, there are other table search techniques that focus on finding joinable tables, i.e., given a query table marked with a query column, find the data lake tables that contain a column with overlapping values with the query column. For instance, LSH Ensemble [14] defines joinable table search as a problem of finding data lake tables that have a column having high set containment with the query column. They propose a novel index that approximates set containment computation. JOSIE [5] searches for top-k joinable tables based on exact set containment. AutoJoin [27], on the other hand, determines the fuzzy join between rows of a given pair of joinable tables by employing string transformations such as matching n-grams of cell values and concatenating cell values. PEXESO [28] efficiently searches for the top-k semantically joinable tables. Along with a join on exact values, they also consider fuzzy join, i.e., join on abbreviations and synonyms. DeepJoin [29] is a deep-learning-based technique that also searches for semantically joinable tables. WarpGate [30] is also a system for searching for top-k semantically joinable tables by embedding each column to vector space using pretrained embeddings. They index column embeddings using Locally Sensitive Hashing [31] for efficient search. Recently, Chorus [32] even demonstrated the ability of Large Language Models to identify joinable columns as a task of completing 'pd.merge' command when two tables are presented as pandas dataframe. Our work addresses the task of discovering tables from the data lakes that can be unionable, joinable, and/or subsets of each other. Notably, we specialize data discovery neural models based on a single pretrained model. The advantage of our approach over above mentioned task specific approaches is that, as in natural language processing, much smaller datasets are sufficient to finetune the model for new data discovery tasks; by leveraging the existing tabular knowledge in the pretrained model.

Benchmarks. Several tabular benchmarks exist in the literature [18], [33], [34]. Koutras et al. [35] created benchmarks for schema matching [36] using open data tables and evaluated the existing matching techniques over them. Mudgal et al [37] open-sourced 13 datasets such as the DBLP-Google Scholar author dataset and Walmart-Amazon product datasets for entity matching task [38]. The WDC Web Table corpus [39], containing around 233 million web tables extracted from different web pages, has been widely used for diverse tasks such as Question Answering [12], [40], Semantic Parsing [11], [40], [41], Table Retrieval [42], Table Metadata Prediction [8], [10], [43] and more [44]. Efthymiou et al. [45] used knowledge graphs to create tabular benchmarks for column type prediction and column-to-column binary relationship prediction tasks. The VizNet Benchmark [46] has been used to evaluate column type prediction tasks [47], [48]. GitTables [33] is a large table corpus in the literature. However, they contain web tables that are small entity tables with fewer rows and columns than enterprise tables and do not represent enterprise data lake scenarios [5]. But the most closely related benchmark to our work is the LakeBench collection of datasets [17] for identifying relation between pair of tables (Sec. III-D) Hence, we use that benchmark for finetuning our model. Further, there are publicly available table search benchmarks that are also relevant to our work which we use for illustrating the use of TabSketchFM for data discovery. TUS [4] and SANTOS [6] are table union search benchmarks which we include in our experiments. At the time of writing this paper, there were no search benchmarks for table join search and table subset search, so we generated *Wiki join search* and *Eurostat subset search* benchmarks (Sec. IV-C). However, recently two new benchmarks have appeared in this space. One is manually annotated table join and union search benchmark [49] (also called LakeBench[3]) and another is a semantic table search [50] benchmark that used wikipedia categories and navigational links to establish semantic join and union relation.

III. TabSketchFM

In this section, we detail the architecture of TabSketchFM. Like existing tabular models [10], [12], we pretrain the BERT model [7]. But, unlike them, we adapt the BERT input embeddings to accommodate the numerical inputs.

A. Sketches Preparation

While other models linearize table cells as text, we prepare *sketches*: numerical vectors that capture different features of the table. We use MinHash signature to convert sets of strings to numerical vectors. MinHash is a technique that uses hash functions to map similar sets to similar hash values, allowing for fast estimation of Jaccard similarity between two sets [51].[4] Specifically, we create one table-level sketch called a content snapshot and two types of column-level sketches: numerical sketch and minhash sketches.

Content Snapshot. Recognizing that row information could be crucial in detecting similarity of tables, we create a sketch from the first 10000 rows. We convert each row into a string

[3]Not to confuse with Srinivas et al. [17] LakeBench used for finetuning.

[4]We use datasketch library [52] to compute MinHash.

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:39:19 UTC from IEEE Xplore. Restrictions apply.